1. Memory Management

Definition

Memory management is the process of controlling and coordinating computer memory, assigning portions to programs and processes, and ensuring efficient use of memory resources.

It is mainly handled by the **Operating System (OS)** with support from **hardware (MMU – Memory Management Unit)**.

1.1 Objectives of Memory Management

- **Efficient utilization:** Maximize memory use while avoiding wastage.
- **Protection:** Prevent one process from accessing another's memory.
- **Relocation:** Allow programs to move within memory during execution.
- **Sharing:** Enable controlled sharing of memory among processes.
- **Virtualization:** Provide each process with the illusion of having large, continuous memory.

1.2 Memory Management Techniques

1. **Contiguous Allocation:**
    - Each process is given a single continuous block of memory.
    - *Drawback:* Leads to fragmentation.
2. **Paging:**

- Memory is divided into fixed-size **pages** (logical) and **frames** (physical).
- Eliminates external fragmentation.
- Uses a **page table** to map virtual pages to physical frames.

3. **Segmentation:**
   - Memory divided into variable-sized segments (code, data, stack).
   - More flexible than paging but prone to external fragmentation.
4. **Virtual Memory:**
   - Provides an **illusion of large memory** using disk storage.
   - Implemented with **demand paging**.
   - Allows execution of programs larger than physical RAM.

---

2. Cache Memory

Definition

Cache memory is a small, high-speed memory located between CPU and main memory.

- Purpose: To store frequently accessed instructions/data and reduce average memory access time.
- Technology: Usually built using **SRAM (Static RAM)**, which is faster than DRAM.

2.1 Cache Organization

- **Levels of Cache:**
    - *L1 Cache:* Smallest, fastest, located inside CPU.
    - *L2 Cache:* Larger but slower, may be on-chip or separate.
    - *L3 Cache:* Even larger, shared among multiple cores.
- **Mapping Techniques:**
    - *Direct Mapped:* Each block of memory maps to exactly one cache line. Simple but prone to conflicts.
    - *Fully Associative:* Any block can be stored in any cache line. Flexible but expensive.
    - *Set Associative:* Hybrid of the above. Memory block maps to a set, and cache line chosen within that set.
- **Cache Replacement Policies:**
    - *LRU (Least Recently Used):* Replace block not used recently.
    - *FIFO (First In First Out):* Replace oldest block.
    - *Random:* Replace a random block.
- **Write Policies:**
    - *Write-Through:* Updates written to cache and main memory simultaneously.
    - *Write-Back:* Updates only cache, and memory updated later (more efficient).

---

3. Memory Hierarchy

Definition

A **memory hierarchy** is a structured arrangement of storage systems in a computer, organized by **speed, cost, and size**.

3.1 Structure of Memory Hierarchy

From top (fastest, smallest, most expensive) to bottom (slowest, largest, cheapest):

1. **Registers:**
    - Inside CPU, hold temporary data.
    - Fastest memory, but very limited in size.
2. **Cache Memory:**
    - Very fast SRAM close to CPU.
    - Stores frequently accessed data.
3. **Main Memory (RAM):**
    - Stores programs and data currently in use.
    - Slower than cache, but larger.
4. **Secondary Storage:**
    - Hard drives, SSDs.
    - Non-volatile, large capacity, much slower.
5. **Tertiary Storage:**
    - Backup devices (magnetic tapes, optical disks).
    - Very slow but very large.

3.2 Principle of Locality

Memory hierarchies are based on the **locality principle**:

- **Temporal Locality:** Recently accessed data is likely to be accessed again soon.
- **Spatial Locality:** Data close to recently accessed data is likely to be accessed soon.

Caches exploit locality to improve performance.

3.3 Performance Metrics

- **Hit Rate:** Percentage of memory accesses satisfied by the cache.
- **Miss Penalty:** Time to fetch data from lower-level memory when cache misses.
- **Average Memory Access Time (AMAT):**

AMAT=Hit Time+(Miss Rate×Miss Penalty)AMAT = \text{Hit Time} + (\text{Miss Rate} \times \text{Miss Penalty})AMAT=Hit Time+(Miss Rate×Miss Penalty)

✅ Summary

- **Memory Management** ensures efficient and secure use of memory, using techniques like paging, segmentation, and virtual memory.
- **Cache Memory** is a small, high-speed buffer that reduces CPU–memory bottlenecks, with different mapping, replacement, and write policies.
- **Memory Hierarchy** arranges memory systems from fastest (registers) to slowest (secondary storage), balancing **speed, cost, and capacity** by exploiting the **principle of locality**.